

Efficient Modular Polynomial Multiplier for NTT Accelerator of Crystals-Kyber

Yuma Itabashi^{*†}, Rei Ueno^{*†}, and Naofumi Homma^{*†}

^{*} Tohoku University, 2-1-1 Katahira, Aoba-ku, Sendai-shi, Miyagi, 980-8577, Japan

Email: rei.ueno.a8@tohoku.ac.jp and naofumi.homma.c8@tohoku.ac.jp

[†] CREST, JST, 4-1-8 Honcho, Kawaguchi, Saitama, 332-0012, Japan

Abstract—This paper presents a hardware design that efficiently performs the number theoretic transform (NTT) for lattice-based cryptography. First, we propose an efficient modular multiplication method for lattice-based cryptography defined over Proth numbers. The proposed method is based on a K-RED technique specific to Proth numbers. In particular, we divide the intermediate result into the sign bit and the other absolute value bits and handle them separately to significantly reduce implementation costs. Then, we show a butterfly unit datapath of NTT and inverse INTT equipped with the proposed modular multiplier. We apply the proposed NTT accelerator to Crystals-Kyber, which is lattice-based cryptography, and evaluate its performance on Xilinx Artix-7. The results show that the proposed NTT accelerators achieve up-to 3% and 33% higher area-time efficiency in terms of LUTs and FFs, respectively, than conventional best methods. In addition, the low-latency version of the proposed NTT accelerators achieves a 18% lower-latency with an area-time efficiency (in terms of LUTs, FFs, and DSPs) than the existing fastest method.

Index Terms—post-quantum cryptography, lattice-based cryptography, number theoretic transform, accelerator, k-reduction.

I. INTRODUCTION

Post-Quantum Cryptography (PQC), which can ensure security against cryptanalysis using quantum computers, has been recently receiving significant attention owing to the increase of security risk from large-scale gated quantum computers that compromise conventional public key cryptography [1]. Lattice-based cryptography is a typical candidate of PQC. Among several types of Lattice-based cryptography, cryptography based on the Learning with Errors problem (LWE cryptography) is attracting attention because of its good balance of calculation time, key length, and communication volume. For now, four variations of LWE cryptography have been selected as 3rd round finalists in the PQC standardization process carried out by the National Institute of Standards and Technology (NIST).

In the LWE cryptography, the computational cost of modular multiplication over a polynomial ring $\mathbb{R}_q = \mathbb{Z}_q[x]/\langle f(x) \rangle$ is dominant, where q is the prime number, $f(x)$ is the N -th order polynomials defined by the cryptography, and $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. To improve its efficiency, the number theoretic transform (NTT) corresponding to the fast Fourier transform on a finite field is commonly used. Several studies have demonstrated NTT and INTT (Inverse NTT) accelerators with efficient modular polynomial multiplication. Conventional techniques

of improving modular multiplication are Barrett and Montgomery reductions, which can only be performed by addition, bit-shift, and constant coefficient multiplication operations without any division operation [2]. In recent years, further improvements were shown by eliminating the relatively time-consuming constant coefficient multiplication operations at the expense of extra addition and bit-shift operations [3], [4]. In [3], the Barrett reduction method was improved by replacing the constant coefficient multiplication operations with addition and bit-shift operations. In [4], the congruence formula on the modulus was exploited to replace the constant coefficient multiplication operations with addition and bit-shift operations. In addition, in [5], an architecture based on a dedicated modular multiplier and memory access optimization was proposed.

In contrast, another approach for improving the modular multiplication using the characteristics of certain prime numbers (i.e., Proth numbers) was shown in [6], which is called K-RED. K-RED achieves better performance in software than the conventional modular reduction methods including those using Barrett and Montgomery reductions. However, there is no concrete hardware design that fully utilizes the principle of K-RED except for the one [7].

In this study, we propose a new modular polynomial multiplier that correctly and fully performs modular reduction over the modulus of Proth numbers for all possible inputs. The basic idea is to apply K-RED twice to reduce the intermediate value in the output range. In particular, we divide the intermediate result of the first K-RED into the sign bit (MSB) and the other absolute bits before the second K-RED. Combining the pre-computed look-up table and direct calculation, we can perform the second K-RED correctly with less hardware cost.

We then apply our multiplier to the (Crystals-)Kyber, which is LWE cryptography finalists in the NIST PQC project. We present a hardware design that efficiently executes NTT and INTT used in Kyber. In this study, we present a radix-2 butterfly unit that is adaptable to both NTT and INTT and evaluate three types of NTT/INTT datapath architectures implemented on Xilinx Artix-7 for fair comparison with other designs of previous studies [3]–[5], [7]. The synthesis results show that the proposed NTT accelerators has 3% and 33% lower area-time product for LUTs and FFs, respectively, than conventional best methods. In addition, a radix-4 variant of proposed NTT accelerator, which has the lowest-latency among the proposed

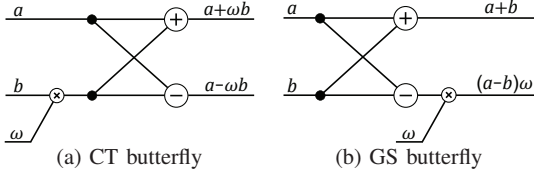


Fig. 1: Block diagrams of butterfly operations.

architectures, achieves a 18% lower-latency with an area-time efficiency (in terms of LUTs, FFs, and DSPs) than the existing fastest method.

II. RELATED WORK

A. polynomial multiplication on $\mathbb{R}_q$

LWE cryptography including Kyber is commonly defined over the polynomial $f(x) = x^N + 1$ (where N is an integer and is given by 256 for Kyber), indicating that the multiplication on $\mathbb{R}_q$ corresponds to the negative wrapped convolution (NWC). To apply NTT to NWC, we consider weighted NTT (NWC NTT) with appropriately weighted coefficients.

Kyber is defined by the operation on $\mathbb{Z}_{3329}[X]/\langle X^{256} + 1 \rangle$. $\mathbb{Z}_{3329}$ has a primitive N -th ($N = 256$) root but a non-primitive $2N$ -th root. Therefore, the 128-point NTT is used instead of the 256-point NTT for Kyber. That is, the 255th-order polynomial is regarded as the 127-th order polynomial, whose the coefficients are given by 1st-order polynomials, and the butterfly operation in NTT and INTT are given as the operations of the 1st-order polynomial.

B. Butterfly operation in NTT/INTT for Kyber

NTT can be implemented at the computational cost of $\mathcal{O}(N \log N)$ for the number of terms N of the polynomial. The butterfly operation is used in the same manner as the Fast Fourier Transform. The N point NTT/INTT usually consists of $\log N$ stages, and each stage performs $N/2$ butterfly operations. For Kyber, NTT/INTT consists of $\log N - 1$ stages because there is no primitive $2N$ -th root.

There are two types of butterfly operations for NTT: Cooley-Tukey (CT) butterfly [8] and Gentleman-Sande (GS) butterfly [9]. Figure 1 shows the block diagrams of CS and GS butterflies. If only one type of butterfly is used in NTT/INTT, a bit-reversal process is required. Pöppelmann et al. showed that such a bit-reversal process could be omitted by using CT and GS butterflies for NTT and INTT, respectively [10]. Therefore, a butterfly unit compatible with both CT and GS butterflies should be implemented while minimizing the amount of resources required.

For efficient modular multiplication, butterfly units are usually pipelined because all butterfly operations in one stage of NTT/INTT can be executed in parallel. The weights of the coefficients are integrated into twiddle factors in each butterfly. See [10] and [11] for more details. INTT requires final multiplications by N^{-1} for all coefficients after all butterfly operations are completed. Zhang et al. proposed a method to multiply the output of each stage (that is, the output of the

Algorithm 1 K-RED

Input: a , **parameter:** m, k
Output: $b \equiv ka \bmod q$
1: $a_0 = a \bmod 2^m$
2: $a_1 = a/2^m$
3: **return** $ka_0 - a_1$

Algorithm 2 K-RED-2x

Input: a , **parameter:** m, k
Output: $k^2a \bmod q$
1: $a_0 = a \bmod 2^m$
2: $a_1 = a/2^m \bmod 2^m$
3: $a_2 = a/2^m$
4: **return** $k^2a_0 - ka_1 + a_2$

butterfly unit) by 2^{-1} [12] to avoid the time-consuming final multiplications. Furthermore, in addition to the architecture that executes radix-2 butterfly operations in parallel [3], [4], [12], the architecture with the radix-4 butterfly unit has been proposed for increasing the throughput and reducing the delay [7], [13], [14].

C. K-RED

K-RED was shown as an efficient modular reduction that can be used when the modulus q is a Proth number (a number represented by $2^m k + 1$ using a small prime number k and a natural number m) [6]. Some LWE cryptography schemes, including Kyber, employ a Proth number (for example, $q = 3329 = 2^8 \cdot 13 + 1$ for Kyber). Algorithm 1 shows the flow of K-RED. When the input a is given as an unsigned integer of α bits, the output b ranges from $-2^{\alpha-m}$ to $2^m k$. The main operations of K-RED are one constant coefficient multiplication and one subtraction at Line 3, whose calculation costs are much smaller than those of conventional operations for Barrett and Montgomery reductions. In addition, the constant coefficient multiplication by a small prime number k can be realized with a smaller number of bit-shifts and additions compared with other recent techniques, as in [3], [4]. Though the output of K-RED is obtained after being multiplied by k , we can multiply the twiddle factors by k^{-1} in advance, and cancel out the effect of multiplying by k at the output.

Algorithm 2 corresponds to two iterations of K-RED, which is called K-RED-2x. The output of K-RED-2x ranges from $-2^m k$ to $2^m k^2 + 2^{\alpha-2m}$ when the input bit length is α . Thus, the output range is smaller than that of K-RED when k is sufficiently small. Note that though the output of K-RED-2x is obtained in the state of being multiplied by k^2 , it is cancelled out in the same manner as above. As an alternative to K-RED-2x, K²-RED, which repeats K-RED twice as it is, has been proposed by Bisheh-Niasar et al. [7] When replacing the constant coefficient multiplication by k with bit-shifts and additions, the calculation costs differ between K-RED-2x and K²-RED depending on the value of the modulus q .

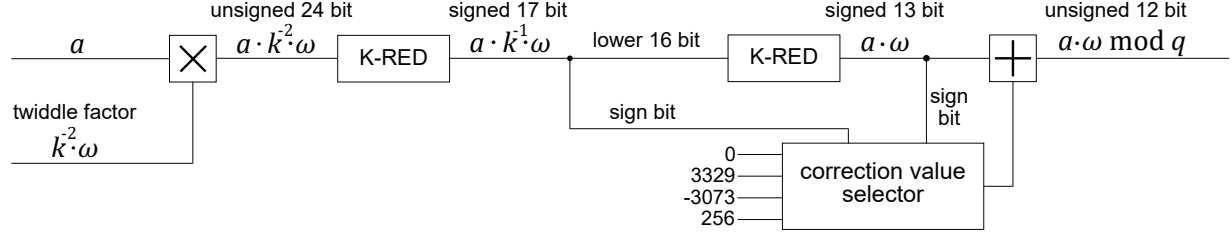


Fig. 2: Proposed modular multiplication.

TABLE I: Correction values in proposed method

correction value	K-RED1 ^a	K-RED2 ^b	K-RED2 ≥ 3073
0	positive	positive	don't care
3329	positive	negative	don't care
-3073	negative	positive	true
256	negative	positive	false
	negative	negative	don't care

^a Output of 1st K-RED

^b Output of 2nd K-RED to the lower 16 bits of K-RED1

TABLE II: Output range and reduction cost

(a) Output range		
modular reduction	min	max
K-RED	-65535	3315
K-RED-2x	-3315	43350
This work	-12	3571

(b) Operation counts (without correction)			
modular reduction	bit-shift	addition	subtraction
K-RED	2	2	1
K-RED-2x	5	6	1
This work	4	4	2

III. PROPOSED DESIGN

A. Modular multiplication based on hybrid K^2 -RED

This study proposes a new modular multiplier based on K-RED. We describe the modular operation for the modulus of Kyber ($q = 3329 = 2^8 \cdot 13 + 1$, that is, $k = 13, m = 8$), where the coefficients and twiddle factors that are the input of butterfly unit(s) are given as unsigned 12-bit integers in the range from 0 to 3328. The output of the butterfly unit must also be an unsigned 12-bit integer because the butterfly unit is used repeatedly and is compatible with all NTT stages. Note, however, that the basic scheme of the proposed method can be applied to other Proth numbers.

Whenever arithmetic (that is, addition, subtraction, and multiplication) operations are performed in the butterfly unit, the output range can temporarily be larger than the input range. After each operation, reduction is performed to keep the output in the range from 0 to 3328. On one hand, reduction after each addition or subtraction is easily performed. The output of unsigned 12-bit addition or subtraction is at most 13 bits. More precisely, the temporal value after addition ranges from 0 to 2×3328 , and the temporal value after subtraction ranges from -3328 to 3328. Then, the reduction can be completed

TABLE III: Operation counts of modular multiplication

modular operation	bit-shift	addition	subtraction
Xing and Li [3]	3	5	4
Yarman et al. [4]	6	7	3
This work	4	5	2

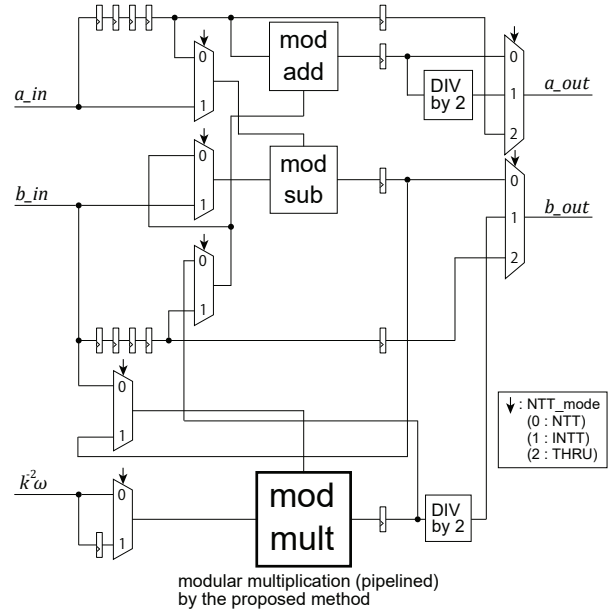


Fig. 3: Butterfly unit.

by subtracting or adding the value of q only once depending on the temporal value.

On the other hand, reduction after multiplication is non-trivial. The bit length of the product can be twice that of the input. After using K-RED or K-RED-2x once, the resulting bit length cannot be sufficiently reduced. The bit length can be reduced by applying K-RED twice (as K^2 -RED in [7]). However, it is unclear how the result is finally reduced/corrected into $[0, 3329)$ due to a lack of description (In fact, the reduction can result in values out of $[0, 3329)$ without correction). Hence, we developed the proposed method of correcting the output range with less computational overhead.

The basic idea of the proposed method is to use a hybrid of look-up table and algorithmic computation to calculate the second K-RED. Specifically, the output obtained by the first

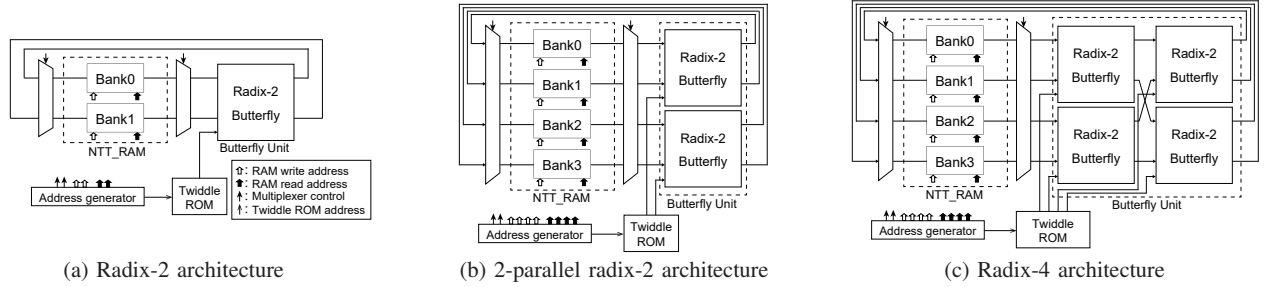


Fig. 4: NTT architectures.

K-RED is divided into two parts: a sign bit and other absolute value bits, and then each part is processed separately. The sign bit is used as one of the selection signals of a table while the absolute bits are reduced by the second K-RED in Algorithm 1. The sign bit derived from the second K-RED is also used as the selection signal. The final output of modular multiplication is given by adding the absolute value of the second K-RED and correction value from the table.

Concretely, we describe the proposed method for the case of Kyber. Figure 2 shows a block diagram of the proposed modular multiplier according to the Kyber parameters. Here, the product of unsigned 12-bit input and a twiddle factor is first obtained using an integer multiplier. The first K-RED is applied to the product of an unsigned 24-bit integer. The output of the first K-RED is a signed 17-bit integer that can be negative. We apply the second K-RED only to the lower 16 bits, and treat the 17th bit, which is the sign bit, as a table selection signal. In the two's complement representation, the sign bit represents the coefficients in the digits of -2^{16} . The result of applying K-RED to -2^{16} is 256 ($= -2^{16} \cdot 13 \bmod 3329$). Therefore, when the first K-RED output is negative (that is, the sign bit is 1), the value of 256 should be added last to obtain the correct output. On the other hand, when the first K-RED output is positive, the result of the second K-RED to the lower 16 bits is used as the correct output. In hardware, the result of applying K-RED to the sign bit ($256 = -2^{16} \cdot 13 \bmod q$) is stored in the look-up table (LUT) in advance (or embedded in the data path), and the calculation is done by selecting the value (0 or 256) to be added according to the value of the sign bit.

The range correction after the second K-RED is also processed in the same manner. The output of the second K-RED is categorised into the sign bit and absolute value bits. The correction is performed by selecting whether the value of 3329 stored in the LUT is added according to the sign bit after the second K-RED. As a result, the correction values that are added at the end is given as Table I. Here, when K-RED1 is negative and K-RED2 is 3073 or more ("true" in the table), $256 (= -2^{16} \cdot 13 \bmod 3329)$ must be added, but the sum exceeds 3329. Therefore, the correction value is set to $-3073 (= -q + 256)$. When K-RED1 is negative, K-RED2 is a positive value less than 3073 ("false" in the table), the correction value is set to 256 because it does not exceed

3329 even if 256 is added. When K-RED1 is positive and K-RED2 is negative, the correction value is set to 3329. No correction value is required when both K-RED1 and K-RED2 are positive. (The existing K^2 -RED [7] that performs K-RED twice without considering the sign bit can produce correct results only if K-RED1 is positive.)

Table II(a) shows the output range when each reduction (K-RED, K-RED-2x, or This work) is applied to an unsigned 24-bit integer. Table II(b) shows the corresponding operation counts before the final correction to obtain the value in the range from 0 to 3328. Here, the operation count is calculated assuming that the multiplication by the constant $k = 13$ is replaced with bit-shift and addition operations. From Tables II, the proposed method can significantly reduce the range while suppressing the operation counts. More concretely, the output of the proposed method is a signed 13-bit integer while those of K-RED and K-RED-2x are signed 17-bit integers. As a result, the proposed method requires only three types of correction values $(-q, 0, q)$ for the final correction, while K-RED and K-RED-2x require 21 and 15 types of correction values, respectively. Table III shows the operation counts (including the final correction) of the proposed and previous methods. We can confirm here that our method achieves fewer operation counts than the previous methods, as in [3], [4].

B. Butterfly unit and NTT architectures

Figure 3 shows the block diagram of the butterfly unit integrating the proposed modular multiplier. The basic structure is the same as those of previous studies [4], [7], where one pipeline register is inserted in each of the multiplier and modular arithmetic unit after multiplication, and between them. The butterfly unit consists of five pipeline stages. The butterfly unit performs the CT butterfly and GS butterfly for NTT and INTT (when the NTT_mode is 0 and 1), respectively. When the NTT_mode is 2, the butterfly unit passes the input without changing it. This mode is used in the following radix-4 architecture, such as in [7], because Kyber does not have a primitive $2N$ -th root. The block "DIV by 2" is to divide the input by 2, and can be realized only by 1-bit right shift and addition [4].

Figure 4 shows an overview of NTT architectures. NTT hardware is roughly composed of NTT_RAM that stores coefficients, Twiddle ROM that stores twiddle factors, butterfly

unit(s), and an address generator that generates addresses in a memory bank that stores coefficients that are input of the butterfly. The architectures considered and evaluated in this paper are (a) radix-2 with one Radix-2 butterfly unit (as shown in Figure 3), (b) 2-parallel radix-2 with two radix-2 butterfly units, and (c) radix-4 with two pairs of two radix-2 butterfly units, according to previous studies [4], [7].

IV. IMPLEMENTATION RESULTS

The proposed NTT hardware was evaluated on Xilinx FPGA Artix-7 (XC7A100TFGG676-3). Table IV shows the number of clock cycles, operating frequency, execution time, LUTs, FFs, DSPs, and BRAM for the three-types of proposed NTT accelerators. For comparison, the results obtained using existing methods are also shown in the table. Both the proposed and existing methods use one DSP block for an integer multiplier in the butterfly unit. For example, 2-parallel radix-2 architecture with two butterfly units has two DSP blocks. From Table IV, we can see that the proposed method can reduce the computational resources while maintaining the execution time [3], [4] for the same number of DSP blocks. Table IV indicates that the area-time product is suppressed compared with previous observations. Thus, we confirm that the proposed NTT accelerator can achieve more efficiency than previous accelerators.

The efficiency of the proposed method is close to that of [7], which is thought to be because a similar multiplier consisting of two successive K-REDs was used for the NTT hardware. Note again that it is unclear how the implementation of [7] corrects the final reduction result. Still, the proposed NTT accelerators (especially the radix-2 variant) achieve a higher area-time efficiency in terms of FFs and DSPs with the comparable number of LUTs, because both the sign bits of the first and second K-RED results are processed by a single look-up table. Although the architecture in [5] has a lower DSP-time efficiency than the proposed accelerators, the proposed accelerators still have advantages in FF and BRAM usages, and can exploit a tradeoff between the latency and the number of DSPs.

In consequence, we confirm that the proposed NTT accelerators achieve up-to 3% and 33% higher area-time efficiency in terms of LUTs and FFs than conventional best methods (i.e., [12] and [7]), respectively. In addition, the low-latency version of the proposed NTT accelerators achieves a 18% lower-latency with an area-time efficiency (in terms of LUTs, FFs, and DSPs) than the existing fastest method (i.e., [7]).

Note that there are some architectures that support several lattice-based cryptographies such as [15]. Such architectures achieve the configurability/applicability at an overhead in performance compared to architectures dedicated to a specific scheme. An efficient extension of our architecture to a configurable one is future work.

V. CONCLUSION

This study presented a new modular polynomial multiplier over Proth numbers, and evaluated NTT/INTT accelerators

equipped with the proposed multiplier for lattice-based cryptography, such as Kyber. The proposed multiplier built in a butterfly unit employs K-RED twice in succession while performing the reduction of sign bits of intermediate values in a table look-up manner. According to previous studies, we evaluated three types of NTT/INTT accelerators with the proposed multiplier through their FPGA implementations. We confirmed that the proposed method is more efficient than existing methods for area-time products. The proposed method to other modular polynomial multiplication over Proth numbers though we focused on the case of Kyber in this study. The application of the proposed method to other lattice-based cryptographic processors still remains a prospect for future studies.

REFERENCES

- [1] P. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.
- [2] E. Alkim, R. Avanzi, J. Bos, L. Ducas, A. de la Piedra, T. Pöppelmann, P. Schwabe, D. Stebila, M. R. Albrecht, E. Orsini, V. Osheter, K. G. Paterson, G. Peer, and N. P. Smart, *NewHope Algorithm Specifications and Supporting Documentation*, 2020.
- [3] Y. Xing and S. Li, "A Compact Hardware Implementation of CCA-Secure Key Exchange Mechanism CRYSTALS-KYBER on FPGA," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2021, no. 2, pp. 328–356, Feb. 2021. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8797>
- [4] F. Yarman, A. C. Mert, E. Öztürk, and E. Savaş, "A Hardware Accelerator for Polynomial Multiplication Operation of CRYSTALS-KYBER PQC Scheme," in *2021 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2021, pp. 1020–1025.
- [5] C. Zhang, D. Liu, X. Liu, X. Zou, B. Liu, and Q. Jiang, "Towards efficient hardware implementation of NTT for Kyber on FPGAs," in *IEEE International Symposium on Circuits and Systems*, 2021.
- [6] P. Longa and M. Naehrig, "Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography," in *Cryptology and Network Security*, S. Foresti and G. Persiano, Eds. Cham: Springer International Publishing, 2016, pp. 124–139.
- [7] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed ntt-based polynomial multiplication accelerator for post-quantum cryptography," in *2021 IEEE 28th Symposium on Computer Arithmetic (ARITH)*, 2021, pp. 94–101.
- [8] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965. [Online]. Available: <http://www.jstor.org/stable/2003354>
- [9] W. M. Gentleman and G. Sande, "Fast Fourier Transforms: For Fun and Profit," in *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference*, ser. AFIPS '66 (Fall). New York, NY, USA: Association for Computing Machinery, 1966, p. 563–578. [Online]. Available: <https://doi.org/10.1145/1464291.1464352>
- [10] T. Pöppelmann, T. Oder, and T. Güneysu, "High-Performance Ideal Lattice-Based Cryptography on 8-Bit ATxmega Microcontrollers," in *Progress in Cryptology – LATINCRYPT 2015*, K. Lauter and F. Rodríguez-Henríquez, Eds. Cham: Springer International Publishing, 2015, pp. 346–365.
- [11] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, "Compact Ring-LWE Cryptoprocessor," in *Cryptographic Hardware and Embedded Systems – CHES 2014*, L. Batina and M. Robshaw, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 371–391.
- [12] N. Zhang, B. Yang, C. Yin, S. Wei, and L. Liu, "Highly Efficient Architecture of NewHope-NIST on FPGA using Low-Complexity NTT/INTT," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 2, pp. 49–72, Mar. 2020. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8544>

TABLE IV: Performance evaluation of NTT/INTT hardware for Crystals-Kyber on Xilinx Artix-7 ($N = 256$ and $q = 3329$)

Work	NTT/INTT Cycles	Freq [MHz]	Time [μ s]	Area				Area-Time Product			
				LUTs	FFs	DSPs	BRAM	LUTs $\times \mu$ s	FFs $\times \mu$ s	DSPs $\times \mu$ s	BRAM $\times \mu$ s
Xing and Li [3]	512/576	161	3.18	1,737	1,167	2	3	5,524	3,711	6.36	9.54
Yarman et al. [4]	904/904	190	4.76	948	352	1	2.5	4,512	1,676	4.76	11.90
Zhang et al. [5]	490/490	257	1.9	609	640	2	4	1,157	1,216	3.8	7.6
Bisheh-Niasar et al. [7]	324/324	222	1.46	801	717	4	2	1,169	1,046	5.84	2.92
This work (radix-2 arch.)	902/902	208	4.34	274	181	1	1.5	1,190	786	4.34	6.51
This work (2 $\times$ radix-2 arch.)	455/455	216	2.11	538	417	2	2.5	1,133	878	4.21	5.27
This work (radix-4 arch.)	268/268	216	1.24	904	811	4	2.5	1,122	1,006	4.96	3.10

- [13] D. T. Nguyen, V. B. Dang, and K. Gaj, "A High-Level Synthesis Approach to the Software/Hardware Codesign of NTT-Based Post-Quantum Cryptography Algorithms," in *2019 International Conference on Field-Programmable Technology (ICFPT)*, 2019, pp. 371–374.
- [14] C. Du, G. Bai, and X. Wu, "High-speed polynomial multiplier architecture for ring-LWE based public key cryptosystems," in *2016 International Great Lakes Symposium on VLSI (GLSVLSI)*, 2016, pp. 9–14.
- [15] K. Derya, A. C. Mert, E. Öztük, and E. Sava, *Microprocessors and Microsystems*, no. 104451, pp. 297–301.